

Recombination in a Self-Modifying Network Search: A Building-Block Validation and a Structural Negative Result

Vasili Gavrilov*

2026

Abstract

Ordinarily a neural network must be handed its shape before it can learn: how many layers, how wide, which activation. The system studied here is handed only the training data and a target error, and finds the rest itself by evolution: its depth and width, whether each layer is dense or a weight-sharing convolutional one, and its own learning rate, momentum, and activation. Onto that mutation-driven search we add *crossover*, recombination of two parents, following the intuition that mixing building blocks should reach a good architecture faster than mutating one lineage. We report an honest pair of results. First, on a problem with genuine separable building blocks, concatenated deceptive trap functions, our one-point crossover beats mutation decisively, and the advantage *grows* with the number of blocks (from $1.8\times$ fewer generations at four blocks to mutation failing outright while crossover still succeeds at sixteen), the textbook fingerprint of a real recombination effect. Second, on the neural architecture search, crossover is on-par with mutation, no better. We diagnose why, with measurements: on our task the error is set far more by the training hyper-parameters than by the architecture, and architecture matters only through a brittle interaction with them; and because each candidate is trained from scratch, the genome carries architecture but not the learned solution, so there is nothing for recombination to transfer. Neural capacity is fungible and its solutions live in weights, not in the architecture genome, which is the structural reason random search is a strong architecture-search baseline and crossover is routinely dropped. We identify *weight inheritance*, carrying a child's trained parts across generations so a useful combination keeps evolving, as the change that would give recombination something to transfer, and mark it as the next step.

1 Introduction

Neural architecture search (NAS) automates a choice a human normally makes: the shape of the network. Its methods differ mainly in the search strategy: reinforcement learning [8], gradient-based relaxation [10], Bayesian optimization, random search, and evolutionary search [12]. Ours is evolutionary: the topology is evolved while each candidate's weights are trained by back-propagation [9]. Ordinarily the architecture and training knobs are arguments the user supplies, `predict(topology, activation, ...)`; the system here is handed only the data and a target error, `self_modifying_predict(data, error)`, and discovers the rest. The engine grew out of the author's 1997 seminar thesis, which derived the general multi-layer delta rule the training still uses [2].

A recurring, humbling result in this field is that *random search is hard to beat* [7, 11]. This paper asks whether *recombination* can do better than mutation at the architecture search:

*Independent researcher. Physics and computer science by education; a professional software engineer with a background in numerical optimization and machine learning, including rule engines and expert systems. Reference implementation: <https://github.com/Anode1/SMBPANN>.

whether crossing two good parents, mixing their building blocks, reaches a target error faster than mutating one. The self-adaptive machinery and the evolution-strategy view we lean on are standard in the optimization literature the author has worked in [3]. The honest answer we reach is two-sided, and the second side is a structural fact about neural architecture search that a positive control makes precise.

2 Method

2.1 The self-modifying search

A network is a feed-forward stack of dense and convolution-like layers. For a dense layer ℓ the forward pass is $a^{(\ell)} = g(W^{(\ell)}a^{(\ell-1)} + b^{(\ell)})$, with g a per-network hidden activation (logistic sigmoid, tanh, or ReLU) chosen by the genome; the output layer is a dense sigmoid. A convolutional layer applies F shared kernels of width K across the previous layer, so its width is derived, $d_\ell = F(d_{\ell-1} - K + 1)$, and its weight count is only FK : the local, weight-sharing structure LeCun introduced for images [6], here a move the search can take. Weights are trained by back-propagation with momentum (the generalized multi-layer delta rule of the 1997 thesis [2]), initialized uniformly in $[-2.4/f_i, 2.4/f_i]$ with f_i the fan-in. All arithmetic is 32-bit float; all randomness flows through a seeded xorshift generator.

A *genome* carries the topology (per hidden layer: dense or convolutional, and its free parameters) and its own training hyper-parameters (learning rate, momentum, activation), plus a self-adaptive mutation rate. One *mutation* makes a single small change: perturb a layer, flip a layer dense $\leftrightarrow$ conv, or add or remove a layer, within depth and width bounds. Reproduction is self-adaptive in the sense of evolution strategies [3]: an offspring inherits and log-normally perturbs its parent’s mutation rate, applies that many mutations, and co-evolves the hyper-parameters. The search keeps a population, retains the best few (elitism), and refills by reproduction, against a matched random-search control that draws the same number of fresh random genomes each generation.

2.2 The crossover operator

To the mutation-only search we add optional *crossover* (`evolve -X`, the percent of offspring made by recombination). A crossover child is built from two elite parents by a one-point splice of their hidden-layer lists, a prefix of one and a suffix of the other, so it inherits building blocks and a new depth from both; its training hyper-parameters are a blend (geometric mean of the learning and mutation rates, arithmetic mean of momentum, one parent’s activation). The child is then mutated as usual. With `-X 0` the operator is inert and the run is byte-identical to the mutation-only search.

3 Experiments

3.1 Does the operator work? A building-block validation

Before asking whether crossover helps the neural search, we check that the operator does what crossover is supposed to do, on a problem built to have separable building blocks. We use concatenated deceptive trap functions: a genome of M blocks of K bits, where a block scores K if all its bits are one, else $K - 1$ minus its number of ones, a deceptive gradient luring a bit-flipping search toward all-zero. The global optimum (all ones) can be reached only by assembling correctly-solved blocks, which is exactly what crossover recombines and mutation cannot climb to. We race the same GA (tournament selection, elitism) with one-point crossover against mutation-only, over 200 seeds, and scan the number of blocks (`make bbtest`).

Table 1: Crossover versus mutation on concatenated deceptive traps ($K = 4$ -bit blocks, population 1000, tournament 3, 200 seeds). As the number of building blocks grows, crossover’s advantage grows: mutation’s cost explodes and it begins to fail outright, while crossover keeps solving. This is the fingerprint of a genuine building-block effect.

blocks M (genome)	mutation-only	crossover + mutation	speedup
4 (16 bits)	9.2 gens, 200/200	5.1 gens, 200/200	1.8×
8 (32 bits)	70.8 gens, 200/200	14.2 gens, 200/200	5×
12 (48 bits)	257 gens, 185/200	22.6 gens, 200/200	11×, more reliable
16 (64 bits)	363 gens, 28/200	42.3 gens, 199/200	mutation fails

Table 1 is decisive. At sixteen blocks mutation-only reaches the global optimum on only 28 of 200 seeds; crossover reaches it on 199, in a fraction of the generations. The operator and the methodology are sound: when separable building blocks exist, crossover finds and combines them, and its advantage scales with the amount of structure.

3.2 Does crossover help the neural search? No

We then run the same on/off comparison on the architecture search: for each of a set of seeds, a fresh synthetic classification task ($d = 8$ inputs, a random-projection sine label), the self-modifying search run twice from the identical initial population, once mutation-only and once with crossover (every offspring recombining two elites), each until its best validation error reaches a target, or a generation cap. Over the paired runs collected, crossover and mutation are indistinguishable: each reached the target on the same fraction of seeds, with the same median generations-to-target and an even head-to-head. Adding recombination did nothing. The next subsection explains why, and the explanation is not that the operator is weak (Table 1), but that the neural search has no building blocks for it to recombine.

3.3 Why: architecture barely matters, and the genome carries no solution

Two measurements on a fixed task tell the story. First, architecture alone does not solve the task at all: trained with default hyper-parameters, every topology we tried, dense and convolutional, wide and deep, scored between 0.25 and 0.59 test-MSE, at or worse than the 0.25 a constant predictor scores. Second, the *hyper-parameters* do the work, and brittly: holding the topology at a small 8,4,1 and varying only the learning rate, momentum, and activation swings the error from 0.09 to 0.65; and with a good hyper-parameter set fixed, varying the topology is non-monotone and narrow, one width learning (8,4,1 $\rightarrow$ 0.09) while its neighbours fail (8,8,1 $\rightarrow$ 0.37, 8,16,1 $\rightarrow$ 0.41). Each architecture wants its own learning rate; evaluated with the wrong one, a fine architecture looks terrible. The landscape is rugged and the two dimensions are entangled, the worst case for a directed search, and crossover of a topology block breaks the co-adaptation that made the block good.

A separate probe removes even that entanglement and still finds no building blocks. A parallel-branch network (make modnas, back-propagation gradient-checked against finite differences, 0/40 mismatches) sums B independent branches into a multi-output prediction, on an additively separable task where each output depends on its own group of inputs, the closest neural analogue of the trap’s independent blocks. Yet the branch count barely changes the error, and no configuration solves the task cleanly, because *neural capacity is fungible*: a single sufficiently-wide branch computes what several specialised ones would, so the architecture does not decompose into independent modules that recombination could assemble.

Underlying both is one fact. In the trap problem the bits *were* the solution, so crossover transferred solutions. In architecture search the genome is architecture and the solution is

weights, retrained from scratch for every candidate, so the genome carries no solution to transfer, and architectural choices are, per above, fungible and interchangeable. There is nothing for recombination to recombine.

3.4 A note on mutation versus random

For completeness: the mutation-only self-modifying search does edge its matched random control on time-to-target, by a small statistically-clear margin over many seeds. But the diagnosis above reframes that edge: since the architecture barely moves the error, the advantage comes from co-evolving the *hyper-parameters* (the self-adaptive learning rate, momentum, activation), not from searching architecture. It is a hyper-parameter result wearing an architecture-search costume, and we do not lean on it.

4 Discussion

The two results compose into one honest statement. Crossover’s building-block advantage is real and provable, and it scales with structure (Table 1). But neural architecture search, as usually posed, *structurally lacks that structure*, for two reasons we have measured: neural capacity is fungible, so architectures do not decompose into independent modules; and solutions live in the trained weights, which a retrain-from-scratch search does not inherit, so the genome carries nothing to transfer. Together these are why random search is a strong NAS baseline and why the field [9] routinely drops crossover. Our contribution is not a new method but a clean demonstration, with a positive control, of exactly when recombination helps and why it usually cannot here.

Limitations. The scale is small and the task synthetic; the neural crossover comparison is preliminary (few paired seeds), though its flatness is consistent with the mechanism the fixed-task measurements establish. Mean squared error stands in for classification error. None of this is hidden.

5 Conclusion and future work

Handed only the problem and a target error, the search discovers architecture, layer type, and training hyper-parameters, and stops when the error is met. Adding recombination did not speed it up, and we now understand structurally why: there are no building blocks in a fungible-capacity, weights-not-inherited architecture genome. The natural fix, and the next step, is *weight inheritance*: let a child carry its parents’ trained parts rather than retrain from zero, so a useful combination that evolution has found keeps evolving instead of restarting, which should both give recombination a solution to transfer and converge faster. That turns the genome from architecture-only into architecture-plus-learned-state, the form in which the building-block intuition, validated in Table 1, can finally apply to networks. Two further steps remain independent of it: a two-dimensional convolution and an image task, where architecture is decisive rather than fungible; and a bridge to a standard tabular NAS benchmark.

Reproducibility

Every result is regenerated from [13] with a single command. The engine builds with `make` (C99, no dependencies) into `smbpann` (worker), `evolve` (search and control, with `-X` for crossover), and `gentask` (task generator); `make ut` runs the unit and gradient-check suite under AddressSanitizer and UBSan. The building-block validation (Table 1) is `make bbtest` then `./bbtest M pop`; the parallel-branch probe is `make modnas`. The neural crossover A/B is `scripts/xover_ab.sh`; the

search and task are controlled by environment variables (DIM, FREQ, TARGET, POP, ELITE, XPCT, and so on). Each run uses a fixed 32-bit xorshift generator, so the numbers are identical on re-running.

References

- [1] D. E. Rumelhart, G. E. Hinton, R. J. Williams. Learning representations by back-propagating errors. *Nature* 323, 1986.
- [2] V. Gavrilov. *Backpropagation Feed-Forward Neural Networks*. Seminar in Artificial Intelligence, Open University of Israel, 1997.
- [3] I. Rechenberg. *Evolutionstrategie*. 1973; H.-P. Schwefel, *Numerical Optimization of Computer Models*, 1981.
- [4] J. H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975 (schemata and building blocks).
- [5] K. Deb, D. E. Goldberg. Analyzing Deception in Trap Functions. *Foundations of Genetic Algorithms* 2, 1993.
- [6] Y. LeCun, B. Boser, J. S. Denker, et al. Backpropagation Applied to Handwritten Zip Code Recognition. *Neural Computation* 1(4), 1989.
- [7] J. Bergstra, Y. Bengio. Random Search for Hyper-Parameter Optimization. *JMLR* 13, 2012.
- [8] B. Zoph, Q. V. Le. Neural Architecture Search with Reinforcement Learning. *ICLR* 2017.
- [9] E. Real, A. Aggarwal, Y. Huang, Q. V. Le. Regularized Evolution for Image Classifier Architecture Search. *AAAI* 2019.
- [10] H. Liu, K. Simonyan, Y. Yang. DARTS: Differentiable Architecture Search. *ICLR* 2019.
- [11] L. Li, A. Talwalkar. Random Search and Reproducibility for Neural Architecture Search. *UAI* 2019.
- [12] T. Elsken, J. H. Metzen, F. Hutter. Neural Architecture Search: A Survey. *JMLR* 20, 2019.
- [13] V. Gavrilov. SMBPANN: a self-modifying backpropagation feed-forward neural network. Source code, <https://github.com/Anode1/SMBPANN>.